Program Optimization

CSE 238/2038/2138: Systems Programming

Instructor:

Birol Gençyılmaz & Zuhal Altuntaş

Slides adapted from Bryant & O'Hallaron's slides

Generally Useful Optimizations

- **Optimizations that you or the compiler should do regardless of processor / compiler**

- **Code Motion**
  - Reduce frequency with which computation performed
    - If it will always produce same result
    - Especially moving code out of loop

```
void set_row(double *a, double *b,
   long i, long n)
{
    long j;
    for (j = 0; j < n; j++)
     a[n*i+j] = b[j];
}
```



```
    long j;
    int ni = n*i;
    for (j = 0; j < n; j++)
     a[ni+j] = b[j];
```

Compiler-Generated Code Motion (-O1)

```
void set_row(double *a, double *b,
   long i, long n)
{
    long j;
    for (j = 0; j < n; j++)
     a[n*i+j] = b[j];
}
```

```
    long j;
    long ni = n*i;
    double *rowp = a+ni;
    for (j = 0; j < n; j++)
     *rowp++ = b[j];
```

```
set_row:
    testq   %rcx, %rcx        # Test n
    jle .L1             # If 0, goto done
    imulq   %rcx, %rdx        # ni = n*i
    leaq(%rdi,%rdx,8), %rdx   # rowp = A + ni*8
    movl$0, %eax                  # j = 0
.L3:                        # loop:
    movsd   (%rsi,%rax,8), %xmm0      # t = b[j]
    movsd   %xmm0, (%rdx,%rax,8)      # M[A+ni*8 + j*8] = t
    addq$1, %rax            # j++
    cmpq%rcx, %rax          # j:n
    jne .L3             # if !=, goto loop
.L1:                        # done:
    rep ; ret
```

Reduction in Strength

- Replace costly operation with simpler one
- Shift, add instead of multiply or divide

  `16*x -->  x << 4`

  - Utility machine dependent
  - Depends on cost of multiply or divide instruction
    - On Intel Nehalem, integer multiply requires 3 CPU cycles
- Recognize sequence of products

```
for (i = 0; i < n; i++) {
  int ni = n*i;
  for (j = 0; j < n; j++)
    a[ni + j] = b[j];
}
```



```
int ni = 0;
for (i = 0; i < n; i++) {
  for (j = 0; j < n; j++)
    a[ni + j] = b[j];
  ni += n;
}
```

Share Common Subexpressions

- Reuse portions of expressions
- GCC will do this with –O1

```
/* Sum neighbors of i,j */
up =    val[(i-1)*n + j  ];
down =  val[(i+1)*n + j  ];
left =  val[i*n     + j-1];
right = val[i*n     + j+1];
sum = up + down + left + right;
```

```
long inj = i*n + j;
up =    val[inj - n];
down =  val[inj + n];
left =  val[inj - 1];
right = val[inj + 1];
sum = up + down + left + right;
```

3 multiplications: *i*n, (i–1)*n, (i+1)*n*

1 multiplication: *i*n*

```
leaq    1(%rsi), %rax  # i+1
leaq    -1(%rsi), %r8  # i-1
imulq   %rcx, %rsi     # i*n
imulq   %rcx, %rax     # (i+1)*n
imulq   %rcx, %r8      # (i-1)*n
addq    %rdx, %rsi     # i*n+j
addq    %rdx, %rax     # (i+1)*n+j
addq    %rdx, %r8      # (i-1)*n+j
```

```
imulq    %rcx, %rsi  # i*n
addq%rdx, %rsi  # i*n+j
movq%rsi, %rax  # i*n+j
subq%rcx, %rax  # i*n+j-n
leaq(%rsi,%rcx), %rcx # i*n+j+n
```

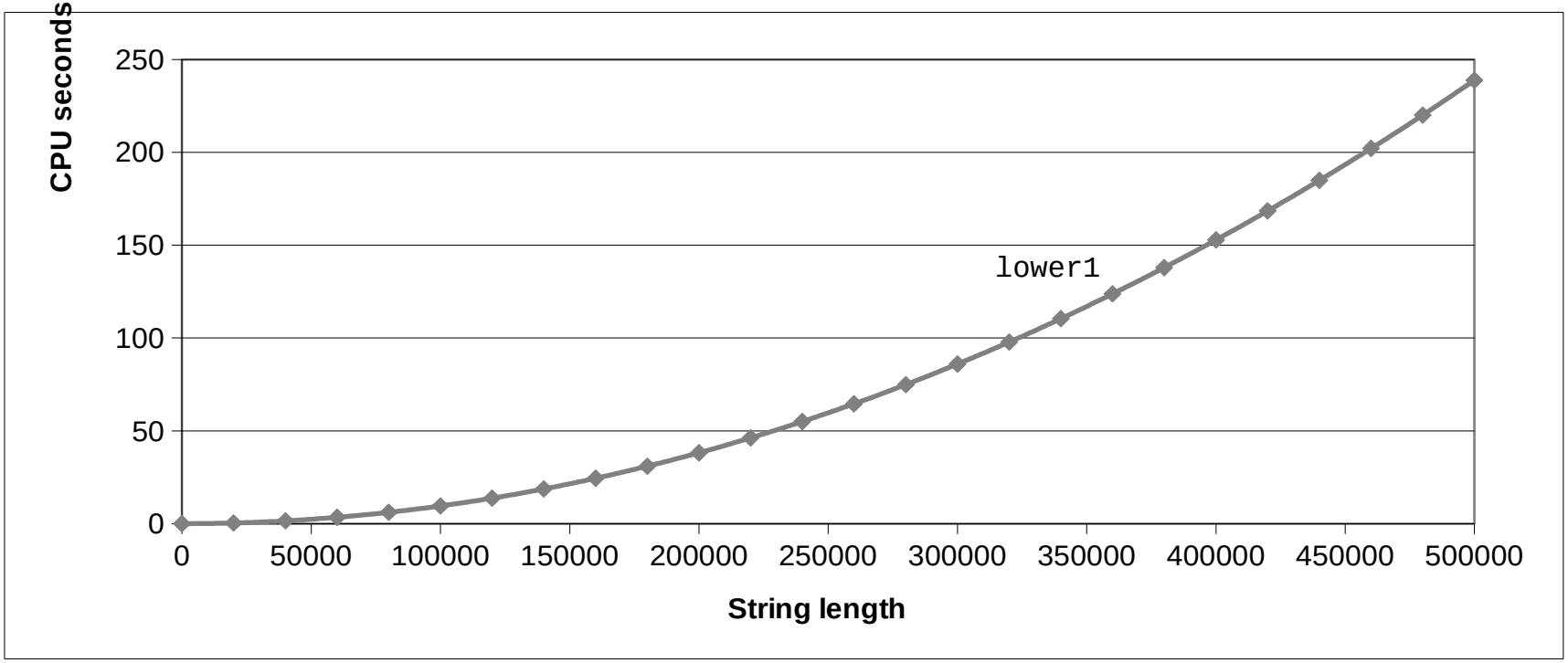
Optimization Blocker #1: Procedure Calls

- **Procedure to Convert String to Lower Case**

```
void lower1(char *s)
{
  size_t i;
  for (i = 0; i < strlen(s); i++)
    if (s[i] >= 'A' && s[i] <= 'Z')
      s[i] -= ('A' - 'a');
}
```

Lower Case Conversion Performance

- Time quadruples when double string length
- Quadratic performance



Convert Loop To Goto Form

```
void lower(char *s)
{
   size_t i = 0;
   if (i >= strlen(s))
     goto done;
 loop:
   if (s[i] >= 'A' && s[i] <= 'Z')
       s[i] -= ('A' - 'a');
   i++;
   if (i < strlen(s))
     goto loop;
 done:
}
```

- `strlen` executed every iteration

Calling Strlen

```
/* My version of strlen */
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
   s++;
   length++;
    }
    return length;
}
```

- **Strlen performance**
  - Only way to determine length of string is to scan its entire length, looking for null character.
- **Overall performance, string of length N**
  - N calls to strlen
  - Require times N, N-1, N-2, …, 1
  - Overall $O(N^2)$ performance

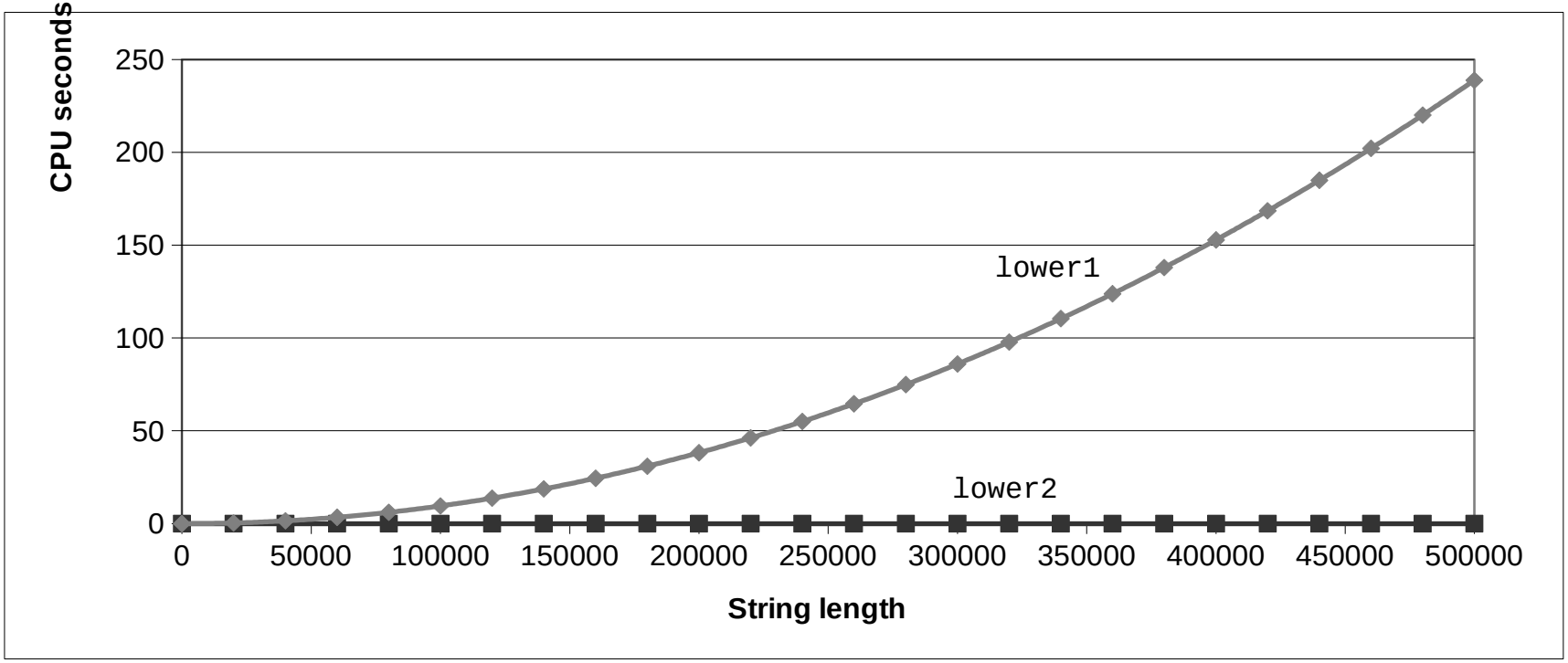
Improving Performance

```
void lower2(char *s)
{
  size_t i;
  size_t len = strlen(s);
  for (i = 0; i < len; i++)
    if (s[i] >= 'A' && s[i] <= 'Z')
      s[i] -= ('A' - 'a');
}
```

- Move call to `strlen` outside of loop
- Since result does not change from one iteration to another
- Form of code motion

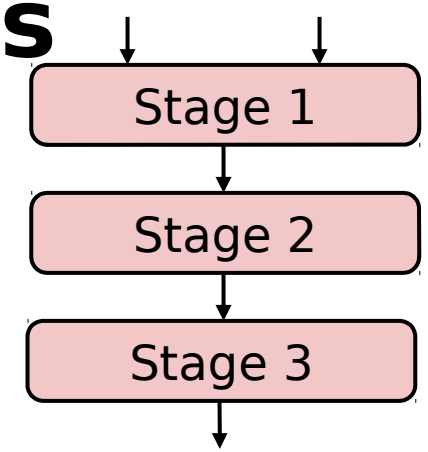
Lower Case Conversion Performance

- Time doubles when double string length
- Linear performance of lower2



Pipelined Functional Units

```
long mult_eg(long a, long b, long c) {
    long p1 = a*b;
    long p2 = a*c;
    long p3 = p1 * p2;
    return p3;
}
```



	Time						
	1	2	3	4	5	6	7
Stage 1	a*b	a*c			p1*p2		
Stage 2		a*b	a*c			p1*p2	
Stage 3			a*b	a*c			p1*p2

- Divide computation into stages
- Pass partial computations from stage to stage
- Stage *i* can start on new computation once values passed to *i+1*
- E.g., complete 3 multiplications in 7 cycles, even though each requires 3 cycles

Loop Unrolling (2x1)

```
void unroll2a_combine(vec_ptr v, data_t *dest)
{
    long length = vec_length(v);
    long limit = length-1;
    data_t *d = get_vec_start(v);
    data_t x = IDENT;
    long i;
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        x = (x OP d[i]) OP d[i+1];
    }
    /* Finish any remaining elements */
    for (; i < length; i++) {
        x = x OP d[i];
    }
    *dest = x;
}
```

- **Perform 2x more useful work per iteration**

How To Compile?

Syntax:

$ gcc -Olevel -S [options] [source files] [-o output file]

option	optimization level	execution time	code size	memory usage	compile time
-O1 or -O	optimization for code size and execution time	-	-	+	+
-O2	optimization more for code size and execution time	--		+	++
-O3	optimization more for code size and execution time	---		+	+++

Examples

- unroll1.c
- unroll2.c
- unroll3.c

objdump

- To display info about object files.
- **Syntax:** objdump [options] objfile

- Examples:

- $ objdump -t bomb : examines the symbol table
- $ objdump -d bomb : disassembles all bomb code

- $ strings bomb : prints all printable strings

gdb : GNU Debugger

- **Syntax:** gdb objfile
- Examine while stepping through your program (registers, the stack, contents of program memory, instruction stream)

- How To Use?
  - run <args> : Run program with arguments
  - break <location> : Set breakpoints
  - disas <fun> : Disassemble a function
  - stepi / nexti : Step in / step trough functions
  - info registers : Print registers' content
  - print $registerName: Print register content